



LaSalle Universidad

Algoritmos Voraces (Greedy)

CURSO: Análisis Y Diseño de Algoritmos

Estudiante: Bran Yames Paz Patatingo

Objetivo

Aplicar la estrategia de algoritmos voraces (Greedy) para resolver problemas de optimización, además de analizar el comportamiento y complejidad de los algoritmos implementados.

Herramientas y/o tecnologías

- git version 2.54.0.windows.1
- 1.123.0 6a44c352bd24569c417e530095901b649960f9f8 x64 (vscode)
- Github
- Programiz Compiler

Mochila Fraccional

Este problema requiere maximizar el valor total transportado sin exceder la capacidad máxima permitida.

Creemos las variables correspondientes con los datos del ejemplo uno.

```
double capacidad = 50;
String[] items = {"A", "B", "C"};
double[] v = {60, 100, 120};
double[] p = {10, 20, 30};
```

Seguidamente creamos “n” para definir la cantidad de iteraciones y el numero de espacios contiguos que debe reservar valor en la memoria heap. Como hay solo un for en esta parte del código se considera que su complejidad es $O(n)$ ya que la cantidad de i no va a crecer exponencialmente. Valor[i] va a tomar el resultado de valor entre peso, lo cual para cuando $i=0$ valor en la posición 0 tomara 6 luego con el siguiente valor hasta terminar la iteración.

```
//creo en heap tres espacios contiguos
int n = items.length;
double[] valor = new double[n];

//valor 0 es 6, valor 1 es 5 y valor 2 es 40
for (int i = 0; i < n; i++) {
    valor[i] = v[i] / p[i];
}
```

Luego, creamos un doble for, el primer for lo usamos para que valor comience desde la posición 0 y el segundo for para que valor en posición j tome el siguiente valor y así entre en el if y se compare, como el resultado del for anterior nos da (siguiendo el ejemplo 1) $\text{valor}[n]=\{6,5,4\}$ nunca entrara al if porque valor en la

posición i para este caso siempre será mayor, pero si valor es 3,5,4 entonces entrara al if para ordenarse del mayor al menor incluyendo cambiar sus datos, como ítems en la posición i es "A" es e ítems en la posición j es "B" estos cambiaran haciendo que ítems[]={ "B", "A", "C"} y lo mismo pasara para valor heap, valor(v) y peso(p).

```
//ordenamos los datos de mayor a menor
for (int i = 0; i < n; i++) {
    for (int j = i + 1; j < n; j++) {
        //el if solo se cumple cuando el valor[i] sea menor al
siguiente dato
        if (valor[i] < valor[j]) {
            //cambio valores de heap
            double temp1 = valor[i]; valor[i] = valor[j];
valor[j] = temp1;
            // cambio las letras
            String temp2 = items[i]; items[i] = items[j];
items[j] = temp2;
            // cambio valor de v y p
            double temp3 = v[i]; v[i] = v[j]; v[j] = temp3;
            double temp4 = p[i]; p[i] = p[j]; p[j] = temp4;
        }
    }
}
```

Creamos un total que nos ayudara a contar los valor(v) que pueden entrar en la capacidad, entramos al if para comprobar si aun nos queda espacio, si es así entonces pasamos al siguiente if que se pregunta si peso en la posición i es menor o igual a la capacidad, si es menor o igual entra y total va a almacenar el valor que ya tenia más valor que está en posición i y se restara la capacidad con el peso en i . Si el peso es mayor a la capacidad entonces parte toma el valor de dividir capacidad restante entre el peso i , y total añadirá el valor en esa misma posición por el valor que toma parte y se sumara con el valor que ya tenia total, por ultimo se crea un boolean para que capacidad sea 0, en caso de que haya mas elementos con su peso y valor, al volver a iterar entrara al primer if y como se cumple saldrá del for porque ya no hay más espacio.

```
double total = 0;

for (int i = 0; i < n; i++) {
    //si la mochilla esta llena entonces salgo del for
    if (capacidad <= 0) break;
    // como el ultimo es 30 solo quedara dsiponible 20 de
capacidad pasa a fraccionarse
    if (p[i] <= capacidad) {
        total += v[i];
        capacidad -= p[i];
    }
}
```

```

        System.out.println(items[i] + " completo");
    } else {
        double parte = capacidad / p[i];
        total += v[i] * parte;
        boolean lleno=true;
        if(lleno == true) capacidad = 0;
        System.out.println("Parte del objeto " + items[i]);
    }
}

```

Output

Ejemplo 1

Datos:

- Capacidad máxima: 50
- Objeto: A, B, C
- Valor: 60, 100, 120
- Peso: 10, 20, 30

```

Objetos seleccionados:
A completo
B completo
Parte del objeto C
Valor total aproxiamdo: 240.0

```

Ejemplo 2

Datos:

- Capacidad máxima: 25
- Objetos: A, B, C
- Valor: 80, 100, 120
- Peso: 20, 10, 30

```

Objetos seleccionados:
B completo
Parte del objeto A
Valor total aproxiamdo: 160.0

```

Cobertura de Antenas

El problema nos indica utilizar la cantidad mínima de antenas para que las casas queden cubiertas con ayuda de su rango de cobertura. El programa recibe las posiciones de casas y un rango de cobertura.

Creamos las variables correspondiente según el ejemplo 1. N toma el tamaño del array.

```
int[] casas = {1, 2, 7, 11, 20, 21, 30};
int R = 5;

int n = casas.length; // n = 7
int cont = 0;
int i = 0;
```

Creamos un while para hacer las iteraciones necesarias hasta que i llegue a n; casaActual toma el valor de casa en la posición i de manera temporal, dondePoner será casaActual mas el rango, cont será la variable para saber la cantidad de antenas usadas, limite es hasta las cordenas de casas que cubre.

```
while (i < n) {

    int casaActual = casas[i];
    int dondePoner = casaActual + R;
    cont++;
    // hasta donde cubre
    int limite = dondePoner + R;
```

El while no termina, pero se genera un dentro que se cumplirá siempre y cuando i sea menor de n y además casas en posición i se menor o igual al limite (11), esta parte usa los valores de casas en i mientras sea menor o igual a 11 para la primera iteración del while principal, para la siguiente iteración i será 5 y limite será 30

```
while (i < n && casas[i] <= limite) {
    System.out.print(casas[i] + " ");
    i++; // saltamos a la siguiente casa ya cubierta
}
```

Output

Ejemplo 1

Datos:

- Casas: 1,2,7,11,20,21,30
- Cobertura: 5

```
beneficiadas: 1 2 7 11
beneficiadas: 20 21 30
antenas utilizadas: 2
```

Ejemplo 2

Datos:

- Casas: 2,4,8,15,18,22
- Cobertura: 3

```
beneficiadas: 2 4 8
beneficiadas: 15 18
beneficiadas: 22
antenas utilizadas: 3
```

Estrategia y Complejidad

Greedy se caracteriza por tomar una decisión optima sin ver el panorama completo, con la esperanza de llegar a la solución óptima.

Mochila Fraccional

- Para cada objeto calculamos el valor que esta aportando, ordenamos los objetos de mayor a menor para recorrer la lista e ir metiendo elementos dentro la mochila, si hay espacio pero no es lo suficiente para que entre todo el valor de un objeto entonces fraccionamos el valor de ese objeto.
- Al haber un doble for n será $n \times n$ porque compara cada elemento con todos los demás por lo cual será $O(n^2)$

Cobertura de Antena

- Primero seleccionamos la primera casa para poner la antena $x+R$ hacia el lado derecho del array para que la antena cubra no solo esa casa, sino algunas más que estén dentro del rango $[x-R, x+R]$, repetimos el proceso con la siguiente casa que no tiene cobertura.
- La complejidad del arreglo si y solo si el array de casas ya está ordenado será $O(n)$ porque no hay for que los ordene y seguirá una frecuencia lineal.